

Práctica 4: Dividir y Conquistar

Contenidos Mínimos

Técnicas de Diseño de Algoritmos / Algoritmos y Estructuras de Datos III

Práctica 4: Dividir y Conquistar

Los ejercicios marcados con * constituyen el subconjunto mínimo de ejercitación sugerido. Los ejercicios que cuentan con sugerencias tienen su correspondiente indicación en la sección *Ayudas* al final del documento.

Ejercicio 1 (MergeSort) *

Dado el algoritmo de mergesort, implementado en el siguiente código Python:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    medio = len(arr) // 2
    mitad_izq = merge_sort(arr[:medio])
    mitad_der = merge_sort(arr[medio:])

    return merge(mitad_izq, mitad_der)
```

y la función auxiliar `merge` :

```
def merge(izq, der):
    mergeados = []
    i = j = 0

    while i < len(izq) and j < len(der):
        if izq[i] < der[j]:
            mergeados.append(izq[i])
            i += 1
        else:
            mergeados.append(der[j])
            j += 1

    mergeados.extend(izq[i:])
    mergeados.extend(der[j:])
    return mergeados
```

1. Identificar qué líneas son el *divide*, cuáles son el *conquer* y cuáles el *combine*.
2. ¿En cuántos subproblemas se divide?

3. ¿De qué tamaño son estos subproblemas?
 4. ¿Cuál es el costo de combinar los resultados de los subproblemas?
 5. Escribir la función $T(n)$ de manera recursiva.
 6. Determinar la complejidad del algoritmo utilizando el Teorema Maestro.
-

Ejercicio 2 (Búsqueda binaria) ★

Dado el algoritmo de búsqueda binaria, implementado en el siguiente código Python:

```
def busqueda_binaria(arr, objetivo, izq=0, der=len(arr) - 1):
    if izq > der:
        return False # Elemento no encontrado

    medio = (izq + der) // 2
    if arr[medio] == objetivo:
        return medio
    elif arr[medio] > objetivo:
        return busqueda_binaria(arr, objetivo, izq, medio - 1)
    else:
        return busqueda_binaria(arr, objetivo, medio + 1, der)
```

1. Identificar qué líneas son el *divide*, cuáles son el *conquer* y cuáles el *combine*.
 2. ¿En cuántos subproblemas se divide?
 3. ¿De qué tamaño son estos subproblemas?
 4. ¿Cuál es el costo de combinar los resultados de los subproblemas?
 5. Escribir la función $T(n)$ de manera recursiva.
 6. Determinar la complejidad del algoritmo utilizando el Teorema Maestro.
-

Ejercicio 3 (Complexity quest) ★

Calcule la complejidad de un algoritmo que utiliza $T(n)$ pasos para una entrada de tamaño n , donde T cumple:

1. $T(n) = T(n - 2) + 5$
2. $T(n) = T(n - 1) + n$
3. $T(n) = T(n - 1) + \sqrt{n}$
4. $T(n) = T(n - 1) + n^2$
5. $T(n) = 2T(n - 1)$
6. $T(n) = T(n/2) + n$
7. $T(n) = T(n/2) + \sqrt{n}$
8. $T(n) = T(n/2) + n^2$
9. $T(n) = 2T(n - 4)$
10. $T(n) = 2T(n/2) + \log n$

11. $T(n) = 3T(n/4)$

12. $T(n) = 3T(n/4) + n$

Intentar estimar la complejidad para cada ítem directamente y luego calcularla utilizando el teorema maestro de ser posible. Para simplificar los cálculos se puede asumir que n es potencia o múltiplo de 2 o de 4 según sea conveniente.

Ejercicio 4 (Izquierda dominante) ★

Escribir un algoritmo con dividir y conquistar que determine si un arreglo de tamaño potencia de 2 es más a la izquierda, donde "más a la izquierda" significa que:

- La suma de los elementos de la mitad izquierda superan los de la mitad derecha.
- Cada una de las mitades es a su vez "más a la izquierda".

Por ejemplo, el arreglo $[8, 6, 7, 4, 5, 1, 3, 2]$ es "más a la izquierda", pero $[8, 4, 7, 6, 5, 1, 3, 2]$ no lo es.

Intentar que su solución aproveche la técnica de modo que la complejidad del algoritmo sea estrictamente menor a $O(n^2)$.

Implementar el algoritmo en su lenguaje de programación favorito.

Ejercicio 5 (Índice espejo) ★

Tenemos un arreglo $a = [a_1, a_2, \dots, a_n]$ de n enteros distintos (positivos y negativos) en orden estrictamente creciente. Queremos determinar si existe una posición i tal que $a_i = i$. Por ejemplo, dado el arreglo $a = [-4, -1, 2, 4, 7]$, $i = 4$ es esa posición.

Diseñar un algoritmo de dividir y conquistar eficiente (cuya complejidad sea de un orden estrictamente menor que lineal) que resuelva el problema. Calcule y justifique la complejidad del algoritmo dado.

Ejercicio 6 (Potencia logarítmica) ★

Encuentre un algoritmo para calcular a^b en tiempo logarítmico en b . Piense cómo reutilizar los resultados ya calculados. Justifique la complejidad del algoritmo dado. Nótese que b puede no ser una potencia de 2.

Ejercicio 7 (Distancia máxima) ★

Dado un árbol binario cualquiera, diseñar un algoritmo de dividir y conquistar que devuelva el tamaño del camino más largo. El algoritmo no debe hacer recorridos innecesarios sobre el árbol.

(Este ejercicio cuenta con una sugerencia en la sección de Ayudas).

Ejercicio 8 (Cazador de falsos) ★

Se tiene una matriz booleana A de $n \times n$ y una operación `conjunciónSubmatriz` que toma $O(1)$ tiempo y que dados 4 enteros i_0, i_1, j_0, j_1 devuelve la conjunción de todos los elementos en la submatriz que toma las filas i_0 hasta i_1 y las columnas j_0 hasta j_1 . Formalmente:

$$\text{conjunciónSubmatriz}(i_0, i_1, j_0, j_1) = \bigwedge_{i_0 \leq i \leq i_1, j_0 \leq j \leq j_1} A[i, j]$$

1. Dar un algoritmo de complejidad temporal estrictamente menor que $O(n^2)$ que calcule la posición de algún `false`, asumiendo que hay al menos uno. Calcular y justificar la complejidad del algoritmo. Notar que n es el ancho de la matriz, no la cantidad de celdas.
 2. Modificar el algoritmo anterior para que cuente cuántos `false` hay en la matriz. Asumiendo que hay a lo sumo 5 elementos `false` en toda la matriz, calcular y justificar la complejidad del algoritmo. Esto se puede lograr con complejidad menor a $O(n^2)$.
-

Ayudas

Ejercicio 7

Para saber el camino más largo de un árbol, posiblemente necesite conocer más que solo los caminos más largos de sus subárboles.